



Department of Computer Science and Engineering
Faculty of Automatic Control and Computer Science
National University of Science and Technology
POLITEHNICA Bucharest



Bachelor Thesis

Titlu lucrare

Supervisor
Assist. Prof. PhD Eng. Ștefan-Dan
Ciocîrlan

Student
El Studento

2026

Contents

- Abstract 1
- 1. Introduction 2
- 2. Background 4
- 3. Architecture 6
- 4. Implementation 8
- 5. Evaluation 11
- 6. Discussion 13
- 7. Conclusions 14
- References 15
- Appendices 15

Abstract

ABSTRACT WRITING GUIDE (sentence by sentence)

1. Domain -one sentence: broad field of the thesis. e.g., “This thesis lies in the field of computer architecture and hardware accelerators.” e.g., “This thesis lies in the field of scientific computing and high-performance numerical methods.”
2. Context -one sentence: specific setting of your work. e.g., “We focus on number-representation systems implemented in hardware.” e.g., “We focus on software libraries for number representations used in large-scale scientific-computing workflows.”
3. Problem -one sentence: the gap or deficiency you address. e.g., “Currently there is no unified hardware library that implements multiple number representations with a common benchmark suite.” e.g., “There is no existing software library that provides diverse number representations together with automated testing and benchmarking.”
4. Importance (optional but recommended) -one sentence: why it matters. e.g., “Choosing an appropriate number format can reduce silicon area, lower energy consumption, and improve computational accuracy.” e.g., “Using the right number format can increase numerical precision, reduce runtime, and lower energy usage.”
5. Solution -one sentence: what you built or contributed. e.g., “We present a parameterizable Chisel library that implements fixed-point, floating-point, posit, and unum formats, together with a shared test-and-benchmark framework.” e.g., “We provide a Scala-based software library integrated into a Jenkins CI pipeline that offers the same set of number representations and automatically runs correctness and performance tests.”
6. Alternative solutions (optional) -one sentence: what exists and what it lacks. e.g., “Existing tools such as FloPoCo focus mainly on floating-point and lack support for newer formats like posits.” e.g., “General-purpose libraries (e.g., NumPy, Julia’s standard library) provide many formats but do not enforce automated testing or CI integration for reproducible evaluation.”
7. Key result -one sentence: most important outcome of your evaluation. e.g., “In our FPGA experiments, the posit-based multiplier achieved 30% lower error than IEEE-754 at comparable resource usage.” e.g., “Benchmarking shows that, for a climate-model kernel, the software posit implementation reduces mean absolute error by 25% versus double-precision while keeping runtime within 5%.”
8. Conclusion -one sentence: what your work enables. e.g., “The resulting benchmark suite lets hardware designers select number representations based on concrete trade-offs between accuracy, area, and power.” e.g., “The library and Jenkins workflow give scientific-computing teams an easy way to experiment with alternative number formats and quantify their impact on correctness and cost.”
9. Impact on the field (optional) -one sentence: broader influence. e.g., “This can lower production costs for numerical accelerators and encourage wider adoption of efficient,

application-specific formats.” e.g., “It promotes more reproducible scientific software by making number-representation choice a first-class, testable concern in CI pipelines.”

Replace the block below with your 7-9 sentence abstract, following the order above.

1. Introduction

INTRODUCTION WRITING GUIDE (paragraph by paragraph)

P1. Domain

- Describe the broad field of the thesis, why it matters, and which aspects are important.
- Example (hardware): “Computer architecture is the foundation of all computing systems; advances in arithmetic units directly affect the performance, energy efficiency, and applicability of modern accelerators.”
- Example (software): “Scientific computing underpins discoveries in physics, chemistry, climate modelling, and engineering; the fidelity and cost of numerical simulations determine how quickly science can progress.”

P2. Context

- Zoom in on the specific setting where the problem appears and explain its relevance to the broader domain.
- Example (hardware): “Within computer architecture, the choice of number representation (fixed-point, floating-point, posit, etc.) determines the trade-off between accuracy, silicon area, and power consumption in hardware accelerators.”
- Example (software): “In scientific-computing workflows, number-representation libraries are called repeatedly inside kernels; their precision and runtime influence the overall reliability and turnaround time of large simulations.”

P3. Problem

- State clearly the gap or deficiency that motivates your work.
- Example (hardware): “There is currently no unified hardware library that offers multiple number representations together with a common benchmark suite, making fair comparison and reuse across projects difficult.”
- Example (software): “Existing software libraries provide many number formats but lack integrated automated testing and continuous-integration support, which hinders reproducible evaluation in large scientific-computing projects.”

P4-P6. Importance (2-3 paragraphs)

- Explain why solving the problem matters, using concrete impacts and, if possible, literature examples.
- Example hardware paragraph 4 (accuracy & performance): “An inappropriate number format can introduce unacceptable error in signal-processing pipelines or waste precious silicon area on under-utilized functional units.”

- Example hardware paragraph 5 (energy & cost): “Reducing the bit-width or adopting a more efficient format lowers dynamic power and static leakage, directly decreasing the energy per operation and the overall chip cost.”
- Example hardware paragraph 6 (literature/example): “Recent work on neural-network accelerators shows that switching from IEEE-754 float to posits can improve inference accuracy by up to 15 % while cutting energy by 20 %.” [web:119]
- Example software paragraph 4 (precision & reliability): “In climate-model simulations, floating-point rounding errors can accumulate and mask real physical signals, leading to misleading predictions.”
- Example software paragraph 5 (runtime & resources): “More compact representations reduce memory bandwidth and cache pressure, shortening kernel execution and allowing larger problem sizes on the same hardware.”
- Example software paragraph 6 (literature/example): “Studies in high-performance computing demonstrate that posit-based arithmetic can cut the time-to-solution of turbulent-flow simulations by 10 % without sacrificing scientific validity.” [web:121]

P7-P8. Alternative solutions (2 paragraphs)

- Briefly discuss what already exists and what it lacks concerning your goal.
- Example hardware paragraph 7: “FloPoCo provides a rich set of floating-point cores but focuses mainly on IEEE-754 and related formats, offering limited support for posit or unum.”
- Example hardware paragraph 8: “Several open-source Chisel libraries implement individual formats (e.g., a fixed-point package) but they do not share a common test harness, making cross-format benchmarking ad-hoc.”
- Example software paragraph 7: “General-purpose libraries such as NumPy (Python) and the Julia standard library supply many number formats, yet they are not coupled to automated CI pipelines, so developers must run correctness and benchmark tests manually.”
- Example software paragraph 8: “Some research prototypes (e.g., a posit package for Julia) provide the arithmetic but lack integration with build systems like Jenkins, which limits their usability in large, collaborative scientific-computing projects.”

P9. Solution

- Describe what you built or contributed in one concise paragraph.
- Example (hardware): “We present a parameterizable Chisel library that implements fixed-point, floating-point, posit, and unum formats, together with a shared test-and-benchmark framework that can be instantiated for any target FPGA or ASIC.”
- Example (software): “We provide a Scala-based software library that encapsulates the same set of number representations and is wired into a Jenkins CI pipeline; the pipeline automatically compiles, runs correctness tests, and collects performance metrics for each format on demand.”

P10. Experiment overview (optional, 1 paragraph)

- Briefly mention the kinds of benchmarks or experiments you will run to evaluate the solution.
- Example (hardware): “To assess the library we synthesize designs for a mid-range FPGA and run a suite of DSP kernels (FIR filter, matrix multiply, CORDIC) measuring area, maximum frequency, and error versus a high-precision software reference.”
- Example (software): “We evaluate the software library using three representative scientific-computing kernels: a climate-model advection step, an n-body gravitational simulation, and a dense linear-algebra benchmark, reporting runtime, energy (via RAPL), and numerical error.”

P11. Contributions

- List 2-4 concrete contributions of the thesis in prose.
- Example (hardware): “The thesis contributes: (i) a reusable Chisel number-representation library; (ii) a common benchmark framework for fair cross-format comparison; (iii) quantitative evaluation of format trade-offs on FPGA targets; and (iv) guidance for hardware designers on selecting formats for accelerator design.”
- Example (software): “The thesis contributes: (i) a Scala software library with multiple number representations; (ii) a Jenkins-based CI workflow that automates testing and benchmarking; (iii) empirical results showing accuracy-runtime-energy trade-offs for scientific kernels; and (iv) a reusable template for integrating numerical experiments into CI pipelines.”

P12. Thesis structure

- One sentence per chapter summarizing what the reader will find.
- Example (hardware): “Chapter 2 reviews the theoretical foundations of number representation and related work; Chapter 3 presents the high-level architecture of the library and benchmark framework; Chapter 4 details the implementation choices and Chisel code generation; Chapter 5 describes the experimental setup and results; Chapter 6 discusses the implications for accelerator design; and Chapter 7 concludes with summary and future work.”
- Example (software): “Chapter 2 covers the background on scientific computing, number representations, and CI concepts; Chapter 3 outlines the proposed software architecture and Jenkins pipeline; Chapter 4 explains the implementation details and build configuration; Chapter 5 presents the evaluation kernels, metrics, and results; Chapter 6 discusses the impact on scientific-computing workflows; and Chapter 7 concludes with a summary and outlook.”

2. Background

BACKGROUND / THEORETICAL FOUNDATION – WRITING GUIDE (paragraph by paragraph)

P1. Domain

- Describe the broad field of the thesis, why it matters, and which aspects are important.

- Example (hardware): “Computer architecture is the foundation of all computing systems; advances in arithmetic units directly affect the performance, energy efficiency, and applicability of modern accelerators.”
- Example (software): “Scientific computing underpins discoveries in physics, chemistry, climate modelling, and engineering; the fidelity and cost of numerical simulations determine how quickly science can progress.”

P2. Context

- Zoom in on the specific setting where the problem appears and explain its relevance to the broader domain.
- Example (hardware): “Within computer architecture, the choice of number representation (fixed-point, floating-point, posit, etc.) determines the trade-off between accuracy, silicon area, and power consumption in hardware accelerators.”
- Example (software): “In scientific-computing workflows, number-representation libraries are called repeatedly inside kernels; their precision and runtime influence the overall reliability and turnaround time of large simulations.”

P3. Problem and Motivation

- State clearly the gap or deficiency that motivates your work and why solving it is important.
- Example (hardware): “There is currently no unified hardware library that offers multiple number representations together with a common benchmark suite, making fair comparison and reuse across projects difficult.” “This gap forces designers to re-implement basic operators for each format, increasing development time and hindering reproducible evaluation.”
- Example (software): “Existing software libraries provide many number formats but lack integrated automated testing and continuous-integration support, which hinders reproducible evaluation in large scientific-computing projects.” “Consequently, developers must manually run correctness and benchmark tests, slowing down experimentation and increasing the risk of inconsistent results.”

P4. Related Work

- Discuss existing solutions (libraries, tools, architectures) and point out their limitations with respect to your goal.
- Example hardware paragraph 1: “FloPoCo provides a rich set of floating-point cores but focuses mainly on IEEE-754 and related formats, offering limited support for posit or unum.”
- Example hardware paragraph 2: “Several open-source Chisel libraries implement individual formats (e.g., a fixed-point package) but they do not share a common test harness, making cross-format benchmarking ad-hoc.”
- Example hardware paragraph 3: “Recent proposals for programmable arithmetic units (e.g., POSH) demonstrate flexibility but lack a standardized evaluation framework.”
- Example software paragraph 1: “General-purpose libraries such as NumPy (Python) and the Julia standard library supply many number formats, yet they are not coupled to automated CI pipelines, so developers must run correctness and benchmark tests manually.”

- Example software paragraph 2: “Some research prototypes (e.g., a posit package for Julia) provide the arithmetic but lack integration with build systems like Jenkins, which limits their usability in large, collaborative scientific-computing projects.”
- Example software paragraph 3: “Workflow tools like GitHub Actions or GitLab CI can run tests, but they do not provide a ready-made library of number representations for direct reuse.”

P5. Research Question / Goal

- State the main question or engineering goal that your thesis will answer.
- Example (hardware): “Which number-representation format(s) give the best trade-off between accuracy, area, and power for a given class of accelerator kernels?”
- Example (software): “How can we provide a reusable, CI-integrated software library that lets scientific-computing teams evaluate and compare diverse number representations with minimal manual effort?”

P6. Thesis Contributions

- Summarize the main contributions of the thesis in prose (2-4 items).
- Example (hardware): “The thesis contributes: (i) a reusable Chisel number-representation library; (ii) a common benchmark framework for fair cross-format comparison; (iii) quantitative evaluation of format trade-offs on FPGA targets; and (iv) guidance for hardware designers on selecting formats for accelerator design.”
- Example (software): “The thesis contributes: (i) a Scala software library with multiple number representations; (ii) a Jenkins-based CI workflow that automates testing and benchmarking; (iii) empirical results showing accuracy-runtime-energy trade-offs for scientific kernels; and (iv) a reusable template for integrating numerical experiments into CI pipelines.”

Replace each block below (the text between *[and]*) with your own paragraph(s) following the order above.

3. Architecture

ARCHITECTURE CHAPTER - WRITING GUIDE (paragraph by paragraph)

P1. Solution overview

- Describe the overall system you are proposing in one or two sentences.
- State what it does, who uses it, and where it runs.
- Example (hardware): “We propose a parameterizable Chisel library that implements multiple number-representation formats together with a shared test-and-benchmark framework, targeting FPGA-based accelerators.”
- Example (software): “We propose a Scala-based software library integrated into a Jenkins CI pipeline that provides the same set of number representations and automatically runs correctness and performance benchmarks for scientific-computing workloads.”

P2. Functional decomposition

- Break the solution into major functional blocks (black boxes).
- For each block, state its responsibility and what it exposes (interfaces, APIs, ports).

- Do NOT describe internal algorithms yet.
- Example (hardware): “The library consists of: • Arithmetic cores (add, multiply, divide, sqrt) for each supported format; • Conversion utilities (between formats and to/from IEEE-754); • Test harness that drives the cores with stimulus vectors and checks results against a reference model; • Benchmark controller that configures the cores, collects area/frequency/power reports from synthesis tools.”
- Example (software): “The software solution comprises: • Number-representation module (immutable data classes + arithmetic ops for each format); • Test suite (unit tests using ScalaCheck / JUnit that compare results against a high-precision decimal reference); • Benchmark module (runs representative kernels, measures wall-clock time, energy via RAPL, and memory usage); • Jenkins pipeline (checks out code, builds the library, runs the test suite, executes the benchmark module, and publishes results).”

P3. Interfaces & communication

- Explain how the blocks talk to each other (buses, function calls, messages, data files, etc.) and what contracts they obey.
- Example (hardware): “Arithmetic cores expose a parameterized Chisel interface (generic data-width, ready/valid handshake). The test harness drives each core via this interface and compares the output to a software reference model using scoreboard-style checking. The benchmark controller writes configuration registers on the cores and reads back performance counters from the synthesis tool’s reports.”
- Example (software): “The number-representation module offers a pure-functional API (methods like `add`, `mul`, `toDouble`). The test suite calls this API directly in Scala. The benchmark module invokes the same API inside the scientific kernels and logs timing/energy data. Jenkins orchestrates everything via shell steps that invoke `sbt test` and `sbt run`.”

P4. Non-functional considerations

- Discuss any quality attributes you considered when designing the architecture (e.g., modularity, extensibility, resource usage, latency, fault tolerance, reproducibility).
- Example (hardware): “Modularity lets us add a new number format by creating a new arithmetic core without touching existing ones. The design targets low latency (single-cycle where possible) and area efficiency (parameterizable width). The test harness ensures functional correctness before any synthesis, and the benchmark controller enables repeatable power/area estimation across formats.”
- Example (software): “The architecture emphasizes reproducibility (same source, same JVM, same Docker image), extensibility (new formats are added by extending the trait and adding test cases), and performance transparency (benchmark results are stored as JSON artifacts for later comparison). Fault isolation is achieved because each test runs in its own JVM process.”

P5. Mapping to implementation & evaluation (optional, 1 paragraph)

- Briefly note how the architectural blocks will be realized in the implementation chapter and what experiments will validate them in the evaluation chapter.

- Example (hardware): “In Chapter 4 we will show the Chisel code for each arithmetic core and the test-harness Scala code; in Chapter 5 we will synthesize the cores on a Xilinx Artix-7 FPGA and report area, Fmax, and power for a set of DSP kernels.”
- Example (software): “In Chapter 4 we detail the Scala implementation of the number-representation traits and the Jenkinsfile; in Chapter 5 we run the library against three scientific kernels (climate advection, n-body, dense linear algebra) and present runtime, error, and energy numbers.”

P6. Diagram (optional)

- If you like, you can insert a simple block diagram here (using `cetz`, `fletcher`, or just an ASCII-art placeholder) to visualize the blocks and their interfaces.
- Example (hardware): **[You may replace this comment with a figure generated by `cetz` or `fletcher` showing the arithmetic cores, test harness, and benchmark controller.]**
- Example (software): **[You may replace this comment with a figure showing the Scala library, test suite, benchmark module, and Jenkins pipeline.]**

4. Implementation

IMPLEMENTATION CHAPTER - WRITING GUIDE (paragraph by paragraph)

P1. Implementation overview

- Briefly restate what you built and in which language / toolchain.
- One or two sentences.
- Example (hardware): “We implemented the Chisel number-representation library and its test-and-benchmark framework using the Chisel 3.5 hardware construction language, targeting Xilinx FPGAs via the Vivado toolchain.”
- Example (software): “We implemented the Scala-based number-representation library and integrated it into a Jenkins CI pipeline using `sbt` as the build tool, Docker for containerised test environments, and the GitHub Enterprise server for version control.”

P2. Technology choices and justification

- Explain why you picked each major technology (language, framework, hardware platform, CI system, etc.) and what alternatives you considered.
- Example (hardware paragraph): “We chose Chisel over plain Verilog because it gives us parameterizable modules, type-safe elaboration, and easy generation of Verilog for FPGA synthesis. We considered SpinalHDL but stayed with Chisel for its richer ecosystem and better integration with the Chipyard toolbox. The Xilinx Artix-7 FPGA was selected because it is readily available in the university lab and provides a good balance of logic density and DSP slices for arithmetic experiments.”
- Example (software paragraph): “We selected Scala (with `sbt`) because it offers functional-programming abstractions, seamless interoperability with Java libraries, and excellent support for property-based testing (ScalaCheck). We evaluated Python but opted for Scala to keep the implementation type-safe and to leverage the existing Chisel code-generation tools. Jenkins was chosen over GitHub Actions or GitLab CI because our department

already maintains a shared Jenkins master with Docker agents, providing reproducible build environments.”

P3. Core algorithms and data structures

- Describe the key algorithms you implemented (e.g., arithmetic operators, conversion routines, test-generation strategies) and the main data structures that hold state.
- Avoid line-by-line code; focus on the idea.
- Example (hardware): “For each supported format we implemented a parameterizable ripple-carry adder, a combinational multiplier using the Baugh-Wooley algorithm for two ‘s-complement numbers, and a CORDIC-based square-root unit. Conversion between formats is performed by first interpreting the bit-pattern as a rational number (sign x mantissa x 2^{exp}) and then re-encoding it in the target format using rounding-to-nearest-even.”
- Example (software): “The library defines an immutable case class `NumRep[F <: Format]` that holds the bit-pattern and provides methods `+`, `-`, `*`, `/`, and `toDouble`. Each format (IEEE-754 binary32, binary64, posit-8, posit-16, unum-like) extends a sealed trait `Format` that supplies the width, exponent size, and encoding/decoding functions. Test data are generated by property-based generators that sample uniformly from the representable range and also include edge-cases (zero, infinity, NaN, smallest subnormal).”

P4. Problems encountered & decisions made

- Narrate any significant obstacles (design bugs, tooling issues, performance surprises) and the decisions you took to resolve them.
- Example (hardware): “During early synthesis we observed that the naïve ripple-carry adder failed to meet timing at 200 MHz. We replaced it with a carry-look-ahead adder (CLA) for widths > 16 bits, which restored timing with only a modest area increase. Another issue was that the Chisel-generated Verilog contained unnamed wires that caused simulation mismatches; we fixed this by explicitly naming all internal nodes using the `suggestName` directive.”
- Example (software): “The first attempt to run the test suite inside Docker containers failed because the JVM could not access the host ‘s `/dev/kvm` for hardware-accelerated benchmarks. We decided to run the benchmarks directly on the Jenkins agent (bare-metal Ubuntu) while keeping the build and unit-test steps inside Docker to preserve reproducibility. This hybrid approach gave us accurate power/energy measurements via RAPL without sacrificing isolated builds.”

P5. Environment & setup instructions

- Provide a concise “how to run” guide: required software versions, hardware, and any configuration steps.
- Example (hardware): “To reproduce the results you need: • Vivado 2023.2 (or newer) for synthesis; • Chisel 3.5.6 and firrtl 1.6; • Scala 2.13.12 and sbt 1.9.8 for the test harness; • A Xilinx Artix-7 FPGA board (e.g., Nexys A7) with at least 2 GB DDR3; • Optional: Xilinx Power Estimator for post-place-and-route power numbers. Clone the repository, run `sbt test` to execute the unit-test harness, then `make synth` to invoke the Vivado synthesis scripts.”

- Example (software): “To reproduce the results you need: • Docker Engine 24.0+; • OpenJDK 17 (for the sbt build); • sbt 1.9.8; • Jenkins 2.452 with the Docker plugin enabled; • A Linux x86-64 machine with RAPL support (Intel i7-12700K or newer) for energy measurements; • Git (≥ 2.40) for cloning. After cloning, execute `./jenkins/jenkinsfile.sh` locally or push to the GitHub-Enterprise repository to trigger the Jenkins pipeline, which will publish the test and benchmark results as build artifacts.”

P6. Pseudocode & summary tables (optional)

- Present high-level pseudocode for the main algorithms or a summary table of configurable parameters (e.g., format widths, iteration counts).
- Do not dump raw source files.
- Example (hardware pseudocode): “Pseudocode for the parameterizable multiplier:

```
function multiply(a, b, w):
    // w = total bit-width
    // interpret a,b as signed two 's-complement integers
    prod = a * b
    // truncate to w bits (dropping overflow)
    return prod & ((1<<w)-1) "
```

“

- Example (hardware table): “Summary table of supported formats:

Format	Width	Exponent bits	Notes
Fixed-point (Q8.8)	16	-	...
IEEE-754 binary32	32	8	...
Posit-8	8	0	...
Posit-16	16	1	...

“

- Example (software pseudocode): “Pseudocode for the posit addition operation:

```
function addPosit(p, q, es):
    // decode to signed regime, exponent, fraction
    v = decode(p, es); w = decode(q, es);
    r = v + w;
    return encode(r, es); "
```

“

- Example (software table): “Summary table of format parameters:

Format	Total bits	Exponent size (es)	Notes
IEEE-754 binary32	32	8	...
IEEE-754 binary64	64	11	...
Posit-8	8	0	...
Posit-16	16	1	...
Unum-like	32	-	...

5. Evaluation

EVALUATION CHAPTER - WRITING GUIDE (paragraph by paragraph)

P1. Evaluation strategy

- Explain how you will evaluate whether the solution addresses the original problem.
- List the qualities you will measure (accuracy, performance, energy, area, usability, etc.) and why they matter.
- Example (hardware): “To assess whether the Chisel library provides a fair basis for comparing number representations, we measure functional correctness (bit-exact match with a high-precision software reference), resource usage (LUT/FF count, DSP-slice usage, BRAM), maximum clock frequency (Fmax) from synthesis, and estimated power from the Xilinx Power Estimator.”
- Example (software): “To evaluate the software library we verify numerical correctness (error versus a 128-bit decimal reference), runtime (wall-clock time per kernel), energy consumption (via Intel RAPL), and memory footprint (peak RSS).”

P2. Benchmarks & metrics

- Describe the benchmark suite you use, what each benchmark represents, and which metrics you collect for each.
- Example (hardware): “We use a suite of DSP kernels: FIR filter (64-tap), 2-D 8x8 DCT, matrix-vector multiply (64-bit inputs), and a CORDIC-based phase-rotation block. For each kernel we record: area (LUT+FF), DSP-slice usage, BRAM, Fmax (MHz), and estimated dynamic power (mW).”
- Example (software): “We evaluate three representative scientific-computing kernels: climate-model advection step, n-body gravitational simulation, and dense linear-algebra kernel. For each kernel we collect: mean absolute error (vs. 128-bit decimal reference), median runtime (seconds) over 30 runs, average socket power (watts) from RAPL, and peak resident memory (MiB).”

P3. Experimental setup

- Detail the hardware, software, and environmental conditions under which the benchmarks run.
- Include versions, configurations, and any steps taken to ensure fairness.
- Example (hardware): “All designs are synthesized for a Xilinx Artix-7 XC7A200T FPGA (speedgrade -2) using Vivado 2023.2. Target clock is 200 MHz; we enforce a uniform IO-standard (LVCMOS18) and disable unnecessary optimizations (-no bufg). Power estimates are generated with the Xilinx Power Estimator (XPE) using default activity factors (0.1 for registers, 0.2 for logic).”
- Example (software): “Benchmarks run on an Intel Core i7-12700K (Linux 6.8, Ubuntu 24.04) with Intel RAPL enabled for socket-level power measurement. The JVM is OpenJDK 17.0.12; sbt version is 1.9.8. Each benchmark is executed in a fresh Docker container (Ubuntu 22.04, OpenJDK 17) to isolate the build environment, while the benchmark binary

itself runs on the host to access RAPL. We pin the CPU frequency to the base turbo boost (3.6 GHz) and disable hyper-threading to reduce variance.”

P4. Results

- Present the outcome of each benchmark, preferably in tables or figures, and give a short, plain-language interpretation of what the numbers mean.
- You may split this into sub-paragraphs or sub-sections per benchmark.
- Example (hardware): “Table 1 shows the synthesis results for the FIR filter: the posit-8 adder uses 112 LUTs and achieves a Fmax of 340 MHz, while the IEEE-754 binary32 adder needs 158 LUTs and tops out at 285 MHz. The posit-8 version therefore saves $\approx 30\%$ area and allows $\approx 20\%$ higher clock speed. Similar trends appear across the DCT and matrix-vector kernels, with posit-8 consistently delivering the best area-frequency product.” “Figure 2 plots estimated power versus accuracy (mean absolute error) for the CORDIC kernel; the posit-16 operating point lies on the lower-left corner, indicating lower power and lower error than binary32 at the same workload.”
- Example (software): “Table 2 summarizes the climate-advection kernel: the posit-16 implementation incurs a mean absolute error of 1.2×10^{-4} (vs. 3.5×10^{-4} for binary32) while running in 0.84 s (binary32: 0.96 s) and drawing 42 W average socket power (binary32: 48 W). This translates to a $\approx 12\%$ reduction in error, $\approx 13\%$ faster execution, and $\approx 13\%$ lower power.” “Figure 3 shows the energy-vs-error trade-off for the n-body benchmark; the unum-like format dominates the Pareto frontier, delivering the lowest error for a given energy budget.”

P5. Data analysis

- Interpret the patterns across benchmarks. Discuss trade-offs, outliers, and what the results imply for the choice of number representation in different scenarios.
- Example (hardware): “Across all four DSP kernels, the posit-8 format consistently yields the highest Fmax and lowest LUT count, making it attractive for throughput-oriented designs. The posit-16 format, while slightly larger, provides a noticeable accuracy gain (≈ 1 bit extra precision) at a modest area cost ($< 10\%$ increase). The IEEE-754 binary32 format is never optimal on either axis for these kernels, suggesting it is over-engineered for the targeted precision range.”
- Example (software): “For memory-bound kernels (climate advection, n-body) the compact posit-8 and posit-16 formats win because they reduce memory bandwidth and cache pressure, leading to lower runtime and energy. For compute-intensive kernels with high dynamic range (LU decomposition), the unum-like format’s ability to represent very large and very small numbers without overflow/underflow gives it an accuracy edge, even though it uses more bits.”

P6. Evaluation conclusion

- Summarize what the evaluation tells you about the original problem and how your solution addresses it.
- Link back to the research question or goal stated in the background chapter.
- Example (hardware): “The evaluation confirms that a parameterizable hardware library enables fair, reproducible comparison of number representations. Posit-8 emerges as the

best choice for low-latency, low-area accelerator kernels, while posit-16 offers a useful accuracy boost when a few extra LUTs are affordable. This answers our research question: which format(s) give the best accuracy-area-power trade-off for accelerator kernels?”

- Example (software): “The evaluation shows that our Jenkins-integrated software library lets scientific-computing teams swap number representations with a single configuration change and instantly see the impact on correctness, runtime, and energy. Posit-16 provides the best balance for the kernels we tested, confirming that a CI-friendly library can accelerate experimentation with alternative formats in large-scale projects.”

6. Discussion

DISCUSSION CHAPTER - WRITING GUIDE (paragraph by paragraph)

P1. Summary of key findings

- Briefly recap the most important quantitative or qualitative outcomes of your evaluation (no new numbers, just a high-level summary).
- Example (hardware): “Our evaluation showed that the posit-8 format consistently delivers the highest clock frequency and lowest LUT usage across DSP kernels, while posit-16 offers a modest accuracy improvement with a small area overhead.”
- Example (software): “The software evaluation revealed that posit-16 provides the best balance of numerical error, runtime, and energy consumption for the climate-advection and n-body kernels, whereas the unum-like format excels for the LU-decomposition kernel where dynamic range is critical.”

P2. Interpretation of results

- Explain why you observed those outcomes, linking them to the underlying characteristics of the number representations (precision, range, encoding efficiency, etc.).
- Example (hardware): “Posit-8’s tapered accuracy and compact encoding reduce the amount of logic needed for arithmetic, whereas posit-16’s larger fraction gives an extra bit of precision without a proportional increase in routing complexity.”
- Example (software): “Posit-16’s use-of-regime encoding yields efficient representation of numbers near unity, which are common in scientific kernels, while the unum-like format’s variable-size exponent avoids overflow/underflow in the LU decomposition’s large intermediate values.”

P3. Comparison with existing work

- Relate your findings to the related work discussed in the background chapter: do you confirm, extend, or contradict previous claims?
- Example (hardware): “These results agree with earlier FloPoCo studies that posit-8 achieves lower area than IEEE-754 for low-precision DSP[web:119], and they extend the observation to a broader set of kernels and to the posit-16 format.”
- Example (software): “Our findings corroborate recent HPC work that posit-based arithmetic can reduce time-to-solution in turbulent-flow simulations[web:121], and they add the insight that a CI-integrated library makes such experiments repeatable across large teams.”

P4. Implications for the domain

- Discuss how the results affect practitioners or researchers in your broader domain (e.g., hardware designers, scientific-computing teams).
- Example (hardware): “For hardware accelerators, the data suggest that posit-8 should be the default choice for throughput-oriented DSP blocks, while posit-16 is worth considering when a few extra LUTs are available and higher fidelity is needed.”
- Example (software): “Scientific-computing teams can now plug the number-representation library into their Jenkins pipelines and instantly see the trade-off between precision, runtime, and energy, enabling data-driven format selection for climate-modeling, n-body simulations, or linear-algebra workloads.”

P5. Limitations

- Identify any threats to validity, assumptions, or constraints that could affect the generalizability of your results.
- Example (hardware): “Our synthesis targets a single Artix-7 FPGA; results may differ on newer Ultrascale+ or ASIC technologies. The power numbers are estimates from XPE, not measured on silicon.”
- Example (software): “Benchmark kernels are synthetic proxies; real scientific applications may involve different memory access patterns or mixed-precision workflows that could shift the trade-offs.”

P6. Future work

- Propose concrete, feasible extensions that build on your thesis (e.g., new formats, larger benchmarks, hardware-software co-design).
- Example (hardware): “Future work could integrate the library into a full SoC with a RISC-V core, explore posit-32 for higher-dynamic-range kernels, and automate power measurement via on-chip sensors.”
- Example (software): “Future work could add support for stochastic rounding, extend the benchmark suite to include machine-learning inference kernels, and provide a GitHub Action template for broader CI adoption.”

7. Conclusions

CONCLUSIONS CHAPTER - WRITING GUIDE (paragraph by paragraph)

P1. Problem & solution recap

- Briefly restate the original problem and your solution in one or two sentences.
- Example (hardware): “We addressed the lack of a unified hardware library for comparing multiple number representations by delivering a parameterizable Chisel library with a shared test-and-benchmark framework.”
- Example (software): “We solved the missing reproducible evaluation of diverse number representations in scientific-computing projects by providing a Scala-based library integrated into a Jenkins CI pipeline.”

P2. Key results summary

- Highlight the 2-4 most important quantitative or qualitative outcomes of your evaluation (no new details, just the take-away numbers).

- Example (hardware): “Posit-8 gave the highest Fmax and lowest LUT usage across DSP kernels; posit-16 offered a noticeable accuracy gain with < 10 % area overhead.”
- Example (software): “Posit-16 yielded the best balance of error, runtime, and energy for climate-advection and n-body kernels; the unum-like format excelled for LU decomposition where dynamic range is critical.”

P3. Comparison with existing work

- Explain how your results and approach improve upon or differ from the related work discussed in the background chapter.
- Example (hardware): “Unlike FloPoCo, which focuses mainly on IEEE-754 formats, our library supports posit and unum-like formats and provides a common benchmark suite for fair cross-format comparison.”
- Example (software): “Compared to general-purpose libraries such as NumPy, our solution adds automated testing, CI integration, and a ready-made benchmark suite, enabling rapid experimentation in large teams.”

P4. Limitations

- Acknowledge any threats to validity, assumptions, or constraints that affect the generalizability of your findings.
- Example (hardware): “Results are based on synthesis for a single Artix-7 FPGA; power numbers are estimates from XPE, not silicon measurements.”
- Example (software): “Benchmarks use synthetic kernels; real scientific applications may involve different I/O patterns or mixed-precision workflows that could shift the trade-offs.”

P5. Future work

- Propose concrete, feasible extensions that build directly on your thesis (keep this list shorter than the conclusions).
- Example (hardware): “Future work could integrate the library into a full SoC with a RISC-V core, explore posit-32 for higher-dynamic-range kernels, and automate on-chip power measurement.”
- Example (software): “Future work could add stochastic-rounding support, extend the benchmark suite to include machine-learning inference kernels, and provide a GitHub-Action template for broader CI adoption.”

P6. Closing take-away

- End with one memorable sentence that captures the core contribution and its impact.
- Example (hardware): “This work gives hardware designers a practical, reproducible way to select the most efficient number representation for their accelerators.”
- Example (software): “This work equips scientific-computing teams with a CI-friendly toolbox to experiment with and quantify the impact of alternative number representations on correctness and cost.”

References

Appendices

[Fill in the content of this chapter.]